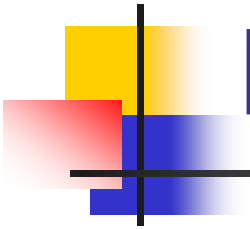


DATA STRUCTURES AND ALGORITHMS

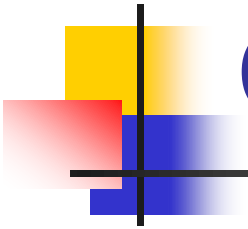
Lecture Notes 10

Prepared by Pradit Pitaksathienkul



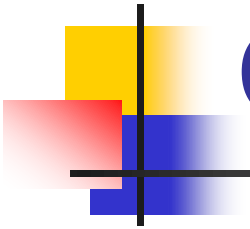
ROAD MAP

- **GRAPH ALGORITHMS**
  - **Definition**
  - **Representation of Graphs**
  - **Topological Sort**



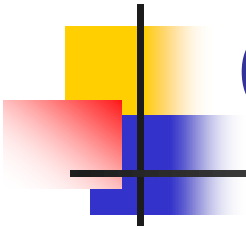
GRAPHS

- A graph is a tuple G=(V,E)
  - V : set of *vertices* / *nodes*
  - E : set of *edges* เส้น
    - each edge is a pair (*v*,*w*), where *v*,*w* ∈ V
  - If the pairs are ordered, then the graph is *directed*
    - directed graphs are sometimes refered as *digraphs*
  - Vertex *w* is adjacent to v iff *(v,w)* ∈ E



GRAPHS

- A path is a sequence of vertices
  - $w_1, w_2, \ldots, w_N$ where $(w_i, w_{i+1}) \in E$ for $1 \leq i < N$
- The length of a path is the number of edges on the path, which is equal to N-1
  - if path contains no edges, length is 0
- If the graph contains an edge (v,v) from a vertex to itself, then the path (v,v) is sometimes refered to a *loop*.
- A simple path is a path such that all vertices are distinct, except that the first and last could be the same

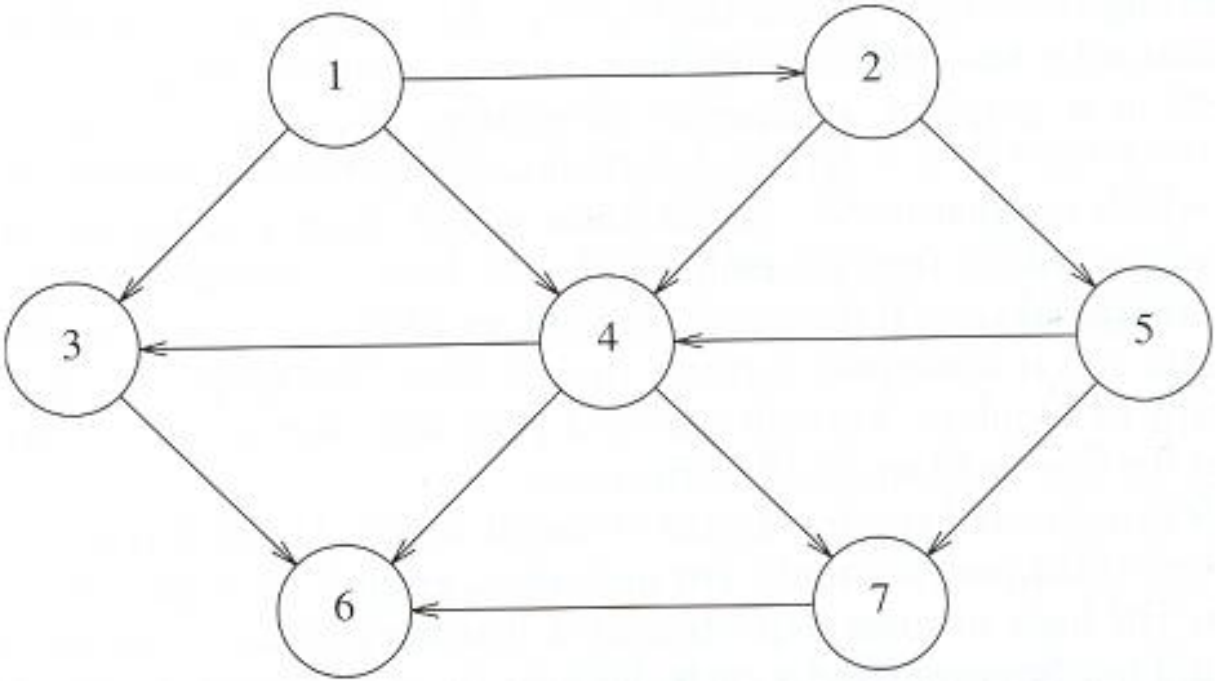


GRAPHS

- A cycle is a path with length ≥ 1 where $w_1 = w_N$
  - Cycle is simple, if the path is simple
  - In an undirected graph edges should be distinct for simple cycle
- A directed graph is acyclic if it has no cycles
  - A directed acyclic graph is refered to DAG
- An undirected graph is connected if there is a **path** between **each pair of vertices**
- A directed graph is strongly connected if there is a path between each pair of vertices
- A directed graph is weakly connected if the underlying undirected graph is connected
- A complete graph is a graph in which there is an **edge** between **every pair of vertices**

Representation of Graphs

- We will consider directed graphs
- We number the vertices, starting at 1.
- The graph below represents 7 vertices and 12 edges



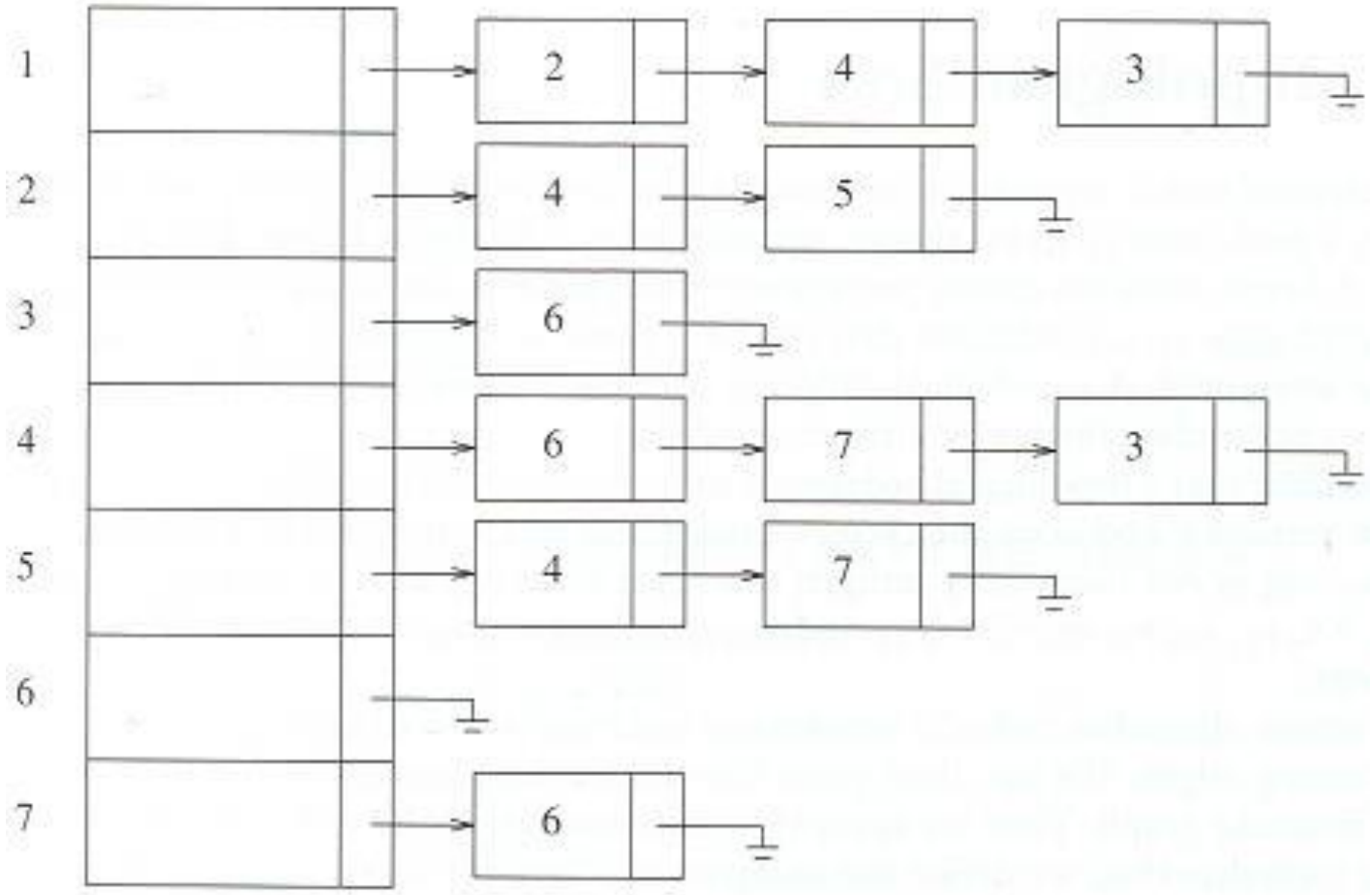
Representation of Graphs

- **Adjacency Matrix Representation**
  - Use a two dimensional array to represent a graph
    - For each edge (u,v) Є E we set A [u][v] = 1
    - A [u][v] = 0 , otherwise
  - If the edge has a weight we set A[u][v] = weight we can use -∞ / ∞ to indicate nonexistent edges
  - Space requirement = $\theta(|v|^2)$ theta
  - An adjacency matrix is an appropriate representation if the graph is dense: $|E| = \theta(|v|^2)$
    - is it true in most applications ?

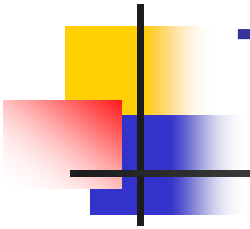
Representation of Graphs

- **Adjacency List Representation**
  - If the graph is not dense (is *sparse*) a better solution is *adjacency list representation*
  - For each vertex, we keep a list of all adjacent vertices.
  - The space requirement is O(|E|+|V|)
  - O อย่างมาก ไม่เกิน 12 + 7= 19ช่อง
- กำหนด Direct Graph มี 10 vertex มี edge 20 เส้น ถามว่า adjacency matrix จะใช้ ? ช่อง

Representation of Graphs



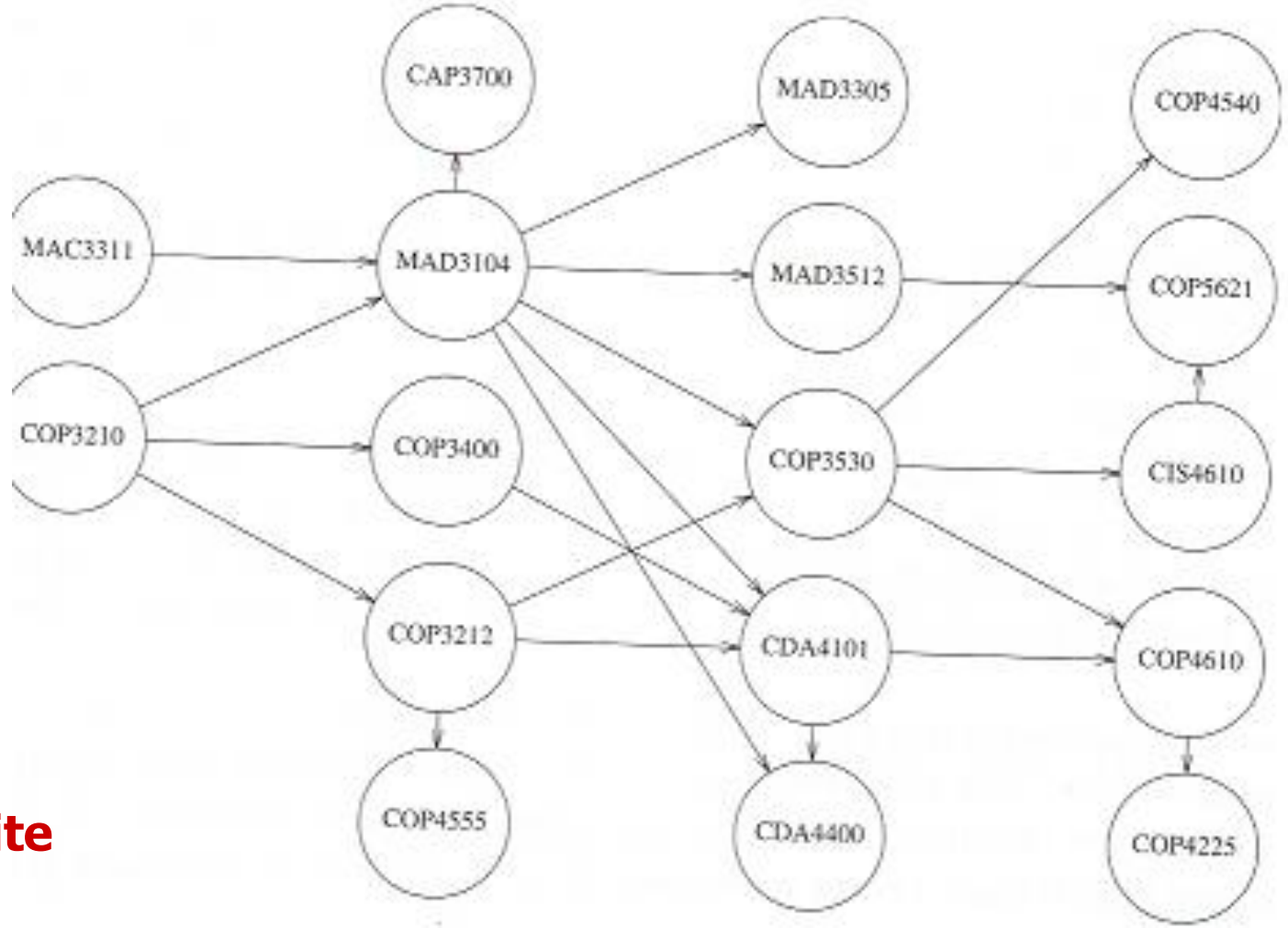
Adjacency list representation of a graph



Topological Sort

- A topological sort is an ordering of vertices in a **directed acyclic graph**
  - If there is a path v_i to v_j, v_j appears after v_i in the ordering
- Example : course prerequisite structure

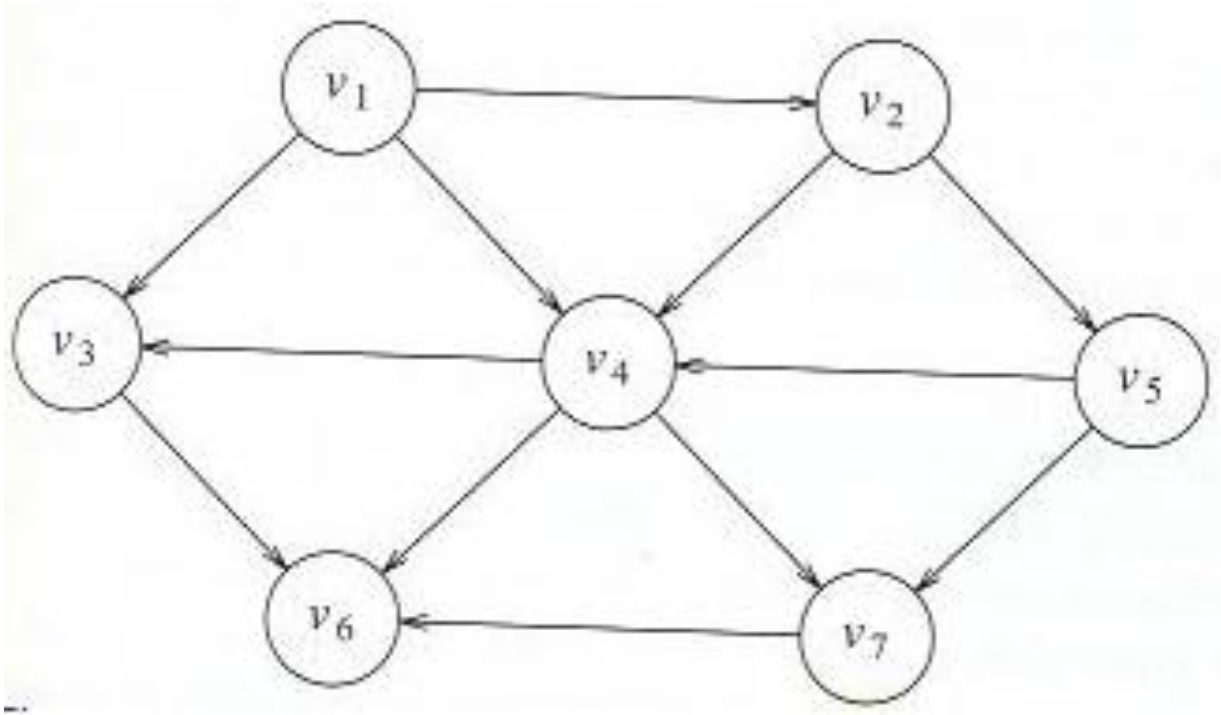
Topological Sort

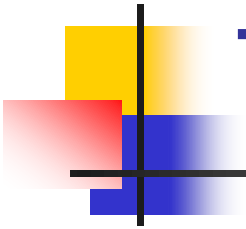


course prerequisite example

Topological Sort

- If a graph has a cycle, a topological sort is not possible
- Ordering is not necessarily unique, any legal ordering will do
- On the example below, v_1, v_2, v_5, v_4, **v_3**, **v_7**, v_6 and
  v_1, v_2, v_5, v_4, **v_7**, **v_3**, v_6 are both topological orderings





Topological Sort

Algorithm

Repeat

find a vertex with no incoming edges

print the node

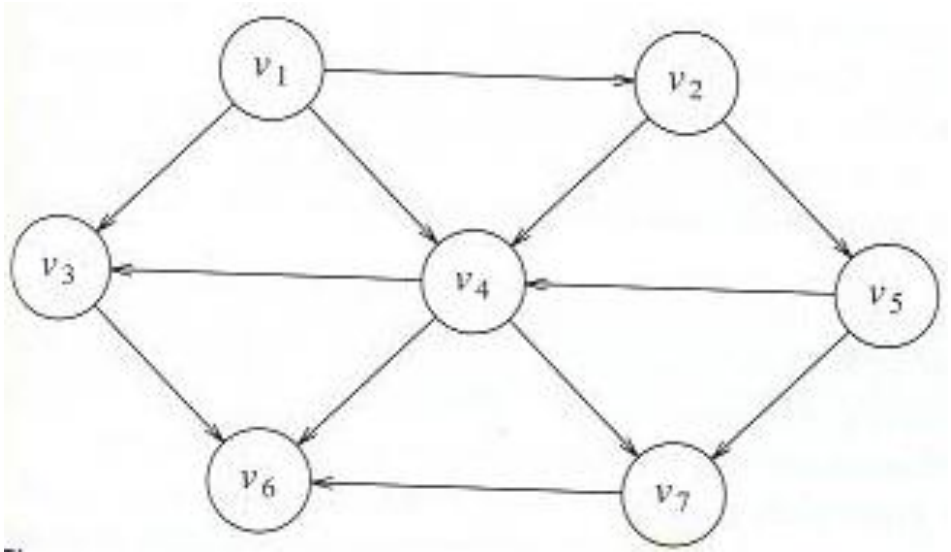
remove it and its edges

Until the graph is empty

How to find a vertex with no coming edges ?

Result of Applying topological sort to the graph below

	Indegree Before Dequeue #						
Vertex	1	2	3	4	5	6	7
v_1	0	0	0	0	0	0	0
v_2	1	0	0	0	0	0	0
v_3	2	1	1	1	0	0	0
v_4	3	2	1	0	0	0	0
v_5	1	1	0	0	0	0	0
v_6	3	3	3	3	2	1	0
v_7	2	2	2	1	0	0	0
Enqueue	v_1	v_2	v_5	v_4	v_3, v_7		v_6
Dequeue	v_1	v_2	v_5	v_4	v_3	v_7	v_6



```
/* Pseudocode ซูโดโค๊ด คำสั่งเทียม to perform topological sort
class Graph():
    def topsort():
        q = Queue(NUM_VERTICES)
        define v, w as vertex

        for each vertex v:
            if v.indegree == 0:
                q.enqueue(v)
        while (!q.isEmpty()):
            v = q.dequeue()
            for each w adjacent to v:
                w.indegree = w.indegree - 1
                if w.indegree == 0:
                    q.enqueue(w)
```